

PROJECT REPORT

Decimal to Binary Converter in Assembly



Course Name	Computer Architecture
Projessor	Mohsen Ali Alawami
Department	Computer Engineering
Student ID	202204495
Name	Changhee Hong
Submission Date	2025.05.28

Contents

1. Project Overview: The Basic Idea.....	2
2. Project Description: Goals, Design, Method, and Results.....	3
2.1. Goals of the Project.....	3
2.2. Project Design.....	4
2.3. Methodology and Implementation Details.....	5
2.3.1. DIS Macro Definition.....	5
2.3.2. Initialize Data/Code Segment.....	5
2.3.3. Display “Enter a decimal number” Message.....	7
2.3.4. Read User Input Character by Character.....	7
2.3.5. Convert ASCII String to Integer.....	9
2.3.6. Store Integer in NO variable.....	10
2.3.7. Start Binary Conversion.....	11
2.3.8. Divide NO by 2, Store Remainder.....	12
2.3.9. Reverse STR1 to STR2.....	13
2.3.10. Display Binary Result.....	14
2.3.11. Program End.....	15
2.4. Project Results.....	16
3. Tool and Software Used.....	17
4. Meaningful Benefits and Findings.....	18

1. Project Overview: The Basic Idea

The "**Decimal to Binary Converter**" project was conceived as a fundamental utility program, developed using Assembly Language, to transform a user-provided decimal (base-10) integer into its equivalent binary (base-2) representation. This project directly stems from the foundational concepts introduced early in our Computer Architecture course, specifically from **Lecture 04 - Computer Number System**, where understanding the internal representation of numbers within a computer system proved to be a particularly insightful and compelling topic. This hands-on exercise was designed to deepen that understanding by delving into the low-level operations of a computer.

The core concept driving this conversion revolves around the **repeated division-by-2 algorithm**. A decimal number is continuously divided by 2, and the remainders of these divisions, when collected in reverse order, collectively form the binary equivalent. For instance, to convert the decimal number 10 to binary:

- $10 / 2 = 5$ (remainder 0)
- $5 / 2 = 2$ (remainder 1)
- $2 / 2 = 1$ (remainder 0)
- $1 / 2 = 0$ (remainder 1)
- Reading the remainders from bottom to top yields 1010, which is the binary representation of 10.

This project focuses on implementing this precise algorithm directly through Assembly Language programming. This approach provides deep insights into **memory management, efficient register usage, arithmetic operations, and basic input/output (I/O) handling** at the machine instruction level. By bypassing the abstractions of higher-level programming languages, this project offers a unique and valuable learning experience in understanding how software truly interacts with hardware.

2. Project Description: Goals, Design, Method, and Results

2.1. Goals of the Project

This project aimed to build a decimal-to-binary converter in Assembly Language, fulfilling both functional and educational objectives.

[Functional Goal]

The primary goal was to create a tool that accurately converts positive decimal integers into their binary form. **A key finding during development was the practical limit of 511 for input values**, a constraint imposed by the 16-bit 8086 architecture's register operations, as further detailed in Section 4.

[Educational Goals (Core Assembly Concepts)]

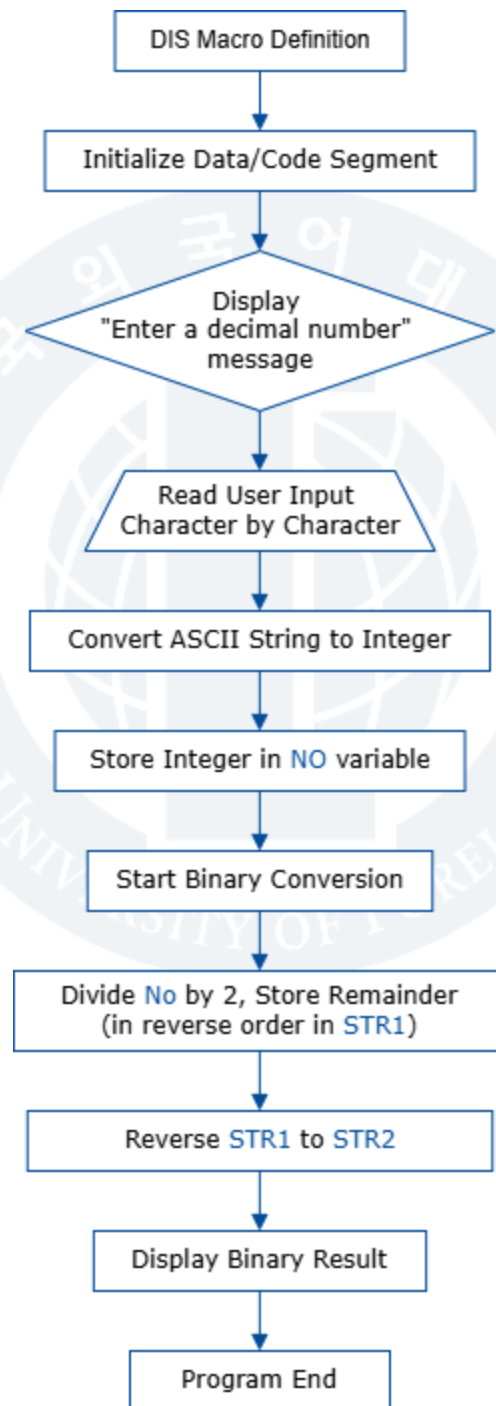
Beyond its function, the project offered a deep dive into fundamental Assembly Language concepts:

- **Basic I/O:** Mastering keyboard input and console output using DOS interrupts.
- **Number System Conversion:** Implementing the decimal-to-binary algorithm at a low level.
- **Data Type Conversion:** Converting ASCII digit strings to numerical values and binary results back to ASCII strings for display.
- **Register & Memory Management:** Efficiently using registers (AX, BX, CX, DX, SI, DI) for data, counters, and addresses, and managing memory for program data.
- **Control Flow:** Implementing jumps, loops, and macro calls to control program execution.
- **Macro Usage:** Learning to define and apply macros for code simplicity and maintainability.

2.2. Project Design

The project's design follows a modular approach, breaking down the complex task into several manageable routines, as depicted in the following flow diagram:

Figure 2.2.1: Flow Diagram



2.3. Methodology and Implementation Details

This section provides a detailed, line-by-line breakdown of the Assembly code,

2.3.1. DIS Macro Definition

```
DIS MACRO STR      ; Defines a macro named DIS that takes one argument, STR.
MOV AH,09H         ; Loads AH register with 09H, which is the DOS service
                   ; number for displaying a string
                   ; When INT 21H is called, AH specifies the service to perform.
LEA DX,STR         ; Loads the effective address (offset) of the
                   ; STR (the string variable passed to the macro) into the DX register.
                   ; The DOS 09H service expects the starting address of
                   ; the string to be displayed in DX
INT 21H           ; Calls DOS Interrupt 21H
                   ; This executes the 09H service specified in AH, causing the string
                   ; pointed to by DX to be displayed on the screen
                   ; The string is displayed until a '$' character is
                   ; encountered, which acts as a terminator
ENDM              ; Marks the end of the DIS macro definition
```

The **DIS** macro is defined to provide a convenient way to display strings on the screen using the DOS **INT 21H** service. In Assembly Language, macros are used to define reusable blocks of code, improving readability and reducing code duplication. Whenever this macro is called, the specified **STR** (string variable) will be printed to the screen.

2.3.2. Initialize Data/Code Segment

```
DATA SEGMENT      ; Marks the beginning of the data segment.
MSG1 DB 0Dh, 0Ah, 'Enter a decimal number: $'
                   ; Defines a message string to prompt the user for input
                   ; (DB: Define Byte)
                   ; 0Dh: Carriage Return (moves cursor to the beginning of the line)
                   ; 0Ah: Line Feed (moves cursor to the next line)
                   ; '$': The string termination character for DOS string output functions
MSG2 DB "BINARY NUMBER IS : $"
                   ; Defines a message to be displayed before the binary result
STR1 DB 20 DUP('$')
                   ; A 20-byte buffer used to temporarily store each bit (0 or 1)
                   ; during the binary conversion process.
                   ; 'DUP($)' initializes all 20 bytes with the '$' character
                   ; A 16-bit (WORD) number can be represented by up to 16 binary bits,
                   ; so 20 bytes is a generous allocation.
STR2 DB 20 DUP('$')
                   ; A 20-byte buffer to copy the bits from STR1 in reverse order,
                   ; forming the correct binary string for display.
                   ; Also initialized with '$' characters.
```

```

NO    DW ?      ; Defines a word (DW: Define Word, 2 bytes) variable to store
                  the decimal integer entered by the user.
                  ; '?' indicates that the variable is uninitialized
                  (only memory space is allocated)

LINE DB 10,13,'$'
                  ; A string for simply printing a newline
                  10 (Line Feed), 13 (Carriage Return)

INBUF DB 6 DUP(0)
                  ; A 6-byte buffer to store the decimal number string input
                  from the keyboard by the user.
                  ; It allows for a maximum of 5 digits plus 1 byte for a null terminator
                  ; 'DUP(0)' initializes all 6 bytes with the value 0

DATA ENDS        ; Marks the end of the data segment

CODE SEGMENT     ; Marks the beginning of the code segment.
    ASSUME DS:DATA,CS:CODE
                  ; Informs the assembler that DS (Data Segment register) will point
                  to the DATA segment,
                  ; and CS (Code Segment register) will point to the CODE segment.
                  ; This is essential for setting up the memory model and allowing
                  the assembler to correctly calculate memory addresses.

INPUT:           ; The starting label for the INPUT routine
                  (program execution begins here due to END INPUT directive)

    MOV AX, DATA ; Loads the starting address (segment address) of
                  the DATA segment into the AX register.

    MOV DS, AX    ; Copies the value from AX into the DS (Data Segment register).
                  ; This crucial step enables the program to access variables
                  defined in the DATA segment (like MSG1, INBUF, etc.).

```

This combined section lays the groundwork for the entire program. The **Data Segment** (`DATA SEGMENT ... DATA ENDS`) explicitly defines all program variables and string constants, allocating memory for them. This includes messages for user interaction (`MSG1`, `MSG2`), buffers for intermediate (`STR1`) and final (`STR2`) binary strings, and a variable (`NO`) to store the converted decimal input.

Following the data definitions, the **Code Segment** (`CODE SEGMENT ... CODE ENDS`) begins, where all executable instructions reside. The `ASSUME` directive is vital; it informs the assembler about the intended segment register assignments, ensuring proper memory addressing. The program's execution technically starts at the `INPUT` label (as specified by `END INPUT`). The initial instructions, `MOV AX, DATA` and `MOV DS, AX`, are fundamental for **initializing the Data Segment Register (DS)**. This setup is crucial because it allows the CPU to correctly locate and access the data defined in the `DATA SEGMENT`, preparing the program for subsequent operations.

2.3.3. Display “Enter a decimal number” Message

This stage involves displaying a prompt message on the screen to inform the user about the required input.

```
INPUT:
    MOV AX, DATA    ; (Conceptually part of 2.3.2, but shown here for continuity)
    MOV DS, AX       ; (Conceptually part of 2.3.2, but shown here for continuity)

    DIS MSG1         ; Calls the DIS macro to display "Enter a decimal number:" message.
                    ; MSG1 includes carriage return and line feed characters
                    ; for proper formatting.
```

The **DIS MSG1** instruction is the core of this stage. It leverages the previously defined **DIS** macro to output the content of the **MSG1** variable ("Enter a decimal number: ") to the user. This serves as a clear prompt, guiding the user on what kind of input is expected in the next phase and enhancing the program's user-friendliness.

2.3.4. Read User Input Character by Character

```
INPUT:
    MOV AX, DATA    ; (Conceptually part of 2.3.2, but shown here for continuity)
    MOV DS, AX       ; (Conceptually part of 2.3.2, but shown here for continuity)

    DIS MSG1         ; (Conceptually part of 2.3.3, but shown here for continuity)

    LEA SI, INBUF     ; Loads the effective address (offset) of the INBUF buffer
                    ; into the SI (Source Index) register.
                    ; SI will be used as a pointer to the current position
                    ; within INBUF where input characters are to be stored.

    XOR CX, CX        ; Clears the CX register by XORing it with itself, setting it to 0.
                    ; CX will serve as a counter for the number of characters
                    ; entered by the user (up to 5 digits).

READ_LOOP:           ; Marks the beginning of the input reading loop.
    MOV AH, 01H      ; Loads AH register with 01H, the DOS service number for reading a
                    ; single character from the keyboard and echoing it to the screen.
    INT 21H          ; Calls DOS Interrupt 21H. The input character is stored
                    ; in the AL register and also displayed on the screen.
    CMP AL, 0Dh      ; Compares the input character in AL with Carriage Return
                    ; (Enter key, ASCII 0Dh).
    JE STR_TO_INT     ; If the Enter key is pressed, jumps to the STR_TO_INT routine
                    ; to convert the string to an integer.
    CMP AL, '0'       ; Compares the input character with ASCII '0'.
```

```

JB READ_LOOP    ; If the character is less than '0' (e.g., control characters),
                  ; it's considered invalid.
                  ; Then jumps back to READ_LOOP to wait for a valid input.
CMP AL, '9'      ; Compares the input character with ASCII '9'.
JA READ_LOOP    ; If the character is greater than '9'
                  ; (e.g., alphabets, special characters), it's considered invalid.
                  ; Then jumps back to READ_LOOP.
MOV [SI], AL     ; If the character is a valid digit ('0' through '9'),
                  ; moves the character from AL to the memory location
                  ; pointed to by SI (within INBUF).
INC SI          ; Increments the SI register by 1, making it point to
                  ; the next available position in INBUF for the next character.
INC CX          ; Increments the CX register (character count) by 1.
CMP CX, 5        ; Compares CX with 5, which is the maximum
                  ; allowed number of digits.
JNE READ_LOOP   ; If CX is not equal to 5, it means not all 5 digits
                  ; have been entered yet.
                  ; Then jumps back to READ_LOOP to continue reading input.
JMP STR_TO_INT  ; If CX has reached 5 (meaning 5 digits have been entered),
                  ; unconditionally jumps to the STR_TO_INT routine,
                  ; even if Enter wasn't pressed.

```

The **READ_LOOP** is central to capturing user input. Before the loop begins, **SI** is initialized to point to the **INBUF** buffer, where characters will be stored, and **CX** is cleared to count the incoming digits. The loop itself repeatedly uses **INT 21H, AH=01H** to read individual characters from the keyboard. For each character, it performs essential **validation**: checking if it's the Enter key (to terminate input) or a valid numeric digit ('0' through '9'). Only valid digits are stored in **INBUF** at the address pointed to by **SI**, which is then incremented. The **CX** counter ensures that the input does not exceed **five digits**, allowing for decimal numbers up to 99,999 to be physically typed. However, it's crucial to note that while the program can accept strings of up to five digits, the subsequent **binary conversion logic (detailed in Section 4. Meaningful Benefits and Findings)** inherently limits the **effective maximum convertible decimal value to 511** due to the 16-bit architecture and 8-bit register constraints in later stages. The loop continues until either the Enter key is pressed or the maximum digit limit is reached, at which point control transfers to the **STR_TO_INT** routine.

2.3.5. Convert ASCII String to Integer

This critical stage converts the ASCII string representation of a decimal number (e.g., '1', '2', '3') stored in **INBUF** into a computable binary integer value (e.g., 123) that the CPU can perform arithmetic operations on.

```
STR_TO_INT:          ; Marks the beginning of the STR_TO_INT routine.
    MOV BYTE PTR [SI], 0
                        ; Stores a null character (0) at the current SI position
                        ; within INBUF.
                        ; This explicitly marks the end of the input string, similar to
                        ; null-termination in C strings,
                        ; allow the conversion loop to properly identify the string's end.
    LEA SI, INBUF      ; Resets SI to point to the beginning address of the INBUF buffer.
                        ; This ensures the conversion process starts from the first digit
                        ; of the input string.
    XOR AX, AX         ; Clears the AX register, which is a 16-bit register,
                        ; used to accumulate the converted decimal integer value.

NEXT_DIGIT:          ; Marks the beginning of the loop for processing each digit.
    MOV BL, [SI]       ; Loads the current character (digit) from the memory location
                        ; pointed to by SI (within INBUF) into the BL register.
    CMP BL, 0          ; Compares the character in BL with a null character (0).
                        ; This check determines if the end of the string has been reached.
    JE STORE_TO_NO     ; If it's a null character, all digits have been processed.
                        ; Jumps to the STORE_TO_NO routine.
    SUB BL, '0'        ; Converts the ASCII code of the current digit
                        ; (e.g., ASCII '5' is 35H) into its actual numeric value (e.g., 5).
                        ; This is done by subtracting the ASCII value of '0' (30H).
    MOV BH, 0          ; Clears the BH register by setting it to 0.
                        ; This is necessary to ensure BX (which combines BH and BL) is
                        ; treated as a 16-bit register for subsequent operations involving
                        ; AX, and only the numeric value in BL is considered.

    ; Calculate AX = AX * 10 + BX (Core Decimal Conversion Logic)
    PUSH AX            ; Pushes the current 16-bit AX value
                        ; (the accumulated decimal value from previous digits)
                        ; onto the stack.
                        ; This saves AX's value before it's modified
                        ; for the multiplication by 10.
    MOV CL, 3          ; Loads CL register with 3. This value specifies
                        ; the number of bits to shift for the SHL instruction.
    SHL AX, CL         ; Shifts the value in AX left by CL (3) bits.
                        ; This effectively multiplies AX by 8 (AX * 8).
    MOV DX, AX         ; Stores the result of (AX * 8) into the DX register.
    POP AX             ; Pops the original value of AX (saved on the stack) back into AX.
                        ; This is the value before the multiplication by 8.
    SHL AX, 1          ; Shifts the value in AX left by 1 bit.
```

```

                This effectively multiplies AX by 2 (AX * 2).
ADD AX, DX      ; Adds the value in DX (AX * 8) to the current value
                in AX (AX * 2).
                ; The result is (AX * 2) + (AX * 8) = AX * 10.
                This efficiently performs multiplication by 10.
ADD AX, BX      ; Adds the numeric value of the current digit (in BX)
                to AX (which now holds AX * 10).
                ; This completes the calculation:
                new AX = (old AX * 10) + current_digit.
                ; Example: If AX was 1 (from digit '1') and current digit is '2',
                AX becomes (1*10)+2 = 12.
                ; Due to AX being a 16-bit register, the maximum decimal value
                that can be accurately accumulated here is 65,535.

INC SI          ; Increments SI by 1 to point to the next digit in INBUF.
JMP NEXT_DIGIT ; Jumps back to the NEXT_DIGIT loop to process the next digit.

```

The **STR_TO_INT** routine converts the ASCII string input into a usable 16-bit integer. It iterates through each character, converts it to its numerical value, and then applies the core decimal conversion formula **AX = AX * 10 + current_digit**. This multiplication by 10 is achieved using bit shifts and addition. It's important to note that since **AX** is a **16-bit register**, the largest decimal number this section can accurately convert and store is 65,535.

2.3.6. Store Integer in NO variable

This stage involves storing the final integer value, which has been successfully converted from the ASCII string in the **STR_TO_INT** routine, into the global variable **NO**. This makes the value accessible for the subsequent binary conversion process.

```

STORE_TO_NO:    ; Marks the beginning of the routine executed
                after all digits have been converted.
MOV NO, AX      ; Moves the final converted decimal integer value
                from the AX register (e.g., 123)
                ; into the pre-defined word (2-byte) variable `NO`.
JMP START       ; Unconditionally jumps to the START label.
                ; This transfers program control to the next major phase:
                converting the decimal number to binary.

```

The **STORE_TO_NO** section executes once the **STR_TO_INT** loop has processed all characters in the input string. The **MOV NO, AX** instruction is used to store the complete

decimal integer from **AX** into the **NO** variable. Since **NO** is a globally accessible variable (defined in the **DATA SEGMENT**), this step ensures that the value is preserved and readily available for the subsequent binary conversion logic (**START** routine). After storing the value, **JMP START** directs the program flow to the next major phase of the project.

2.3.7. Start Binary Conversion

This stage marks the beginning of the core algorithm for converting the decimal integer (stored in the **NO** variable) into its binary string representation.

```
START:                ; Marks the beginning of the START routine
                      ; (entry point for binary conversion logic).
MOV AX,DATA           ; Loads the starting address of the DATA segment into AX.
MOV DS,AX             ; Copies the value from AX into DS.
LEA SI,STR1           ; Loads the effective address of the STR1 buffer
                      ; (where binary bits will be temporarily stored)
                      ; into the SI register.
                      ; SI will serve as a pointer to the current position
                      ; within STR1 for storing binary bits.
MOV AX,NO             ; Loads the decimal value to be converted
                      ; (from the NO variable, e.g., decimal 10)
                      ; into the AX register.
                      ; AX will be the operand for the division operations.
MOV BH,00            ; Initializes the BH register to 0.
                      ; BH will be used as a counter for the number of
                      ; binary bits extracted. (Useful for string reversal later).
MOV BL,2             ; Sets the divisor (the number by which we will divide)
                      ; in the BL register to 2.
                      ; This is fundamental for the repeated division process
                      ; used in decimal to binary conversion.
```

The **START** routine is the entry point for the binary conversion process. It prepares the environment by ensuring **DS** points to the correct data segment. The decimal value from **NO** is loaded into **AX** for arithmetic operations. **STR1** is set up as the destination buffer for binary bits, with **SI** as its pointer. **BH** is initialized to count the extracted bits, and **BL** is set to 2, preparing for the successive division operations that form the core of the binary conversion algorithm.

2.3.8. Divide NO by 2, Store Remainder

This stage implements the core algorithm for decimal-to-binary conversion: **repeated division by 2**. The remainders from these divisions form the binary bits.

```
L1:          ; Marks the beginning of the binary conversion loop.
    DIV BL   ; Divides the current 16-bit value in the AX register
              (which is initially the decimal number, then successive quotients)
              by BL (which holds 2).
              ; After this operation,
              ; The quotient is stored in the 8-bit AL register.
              (e.g., if AX=10, 10/2=5, AL becomes 5)
              ; The remainder of the division is stored in the 8-bit AH register.
              (e.g., if AX=10, 10%2=0, AH becomes 0)
              ; This is a critical point as the 8-bit AL register can only
              hold a maximum value of 255.
    ADD AH, '0' ; Converts the remainder (which is 0 or 1 in AH) into its
                  ASCII character representation ('0' or '1').
                  ; For example, if AH is 0, adding ASCII '0' (30H) makes
                  AH becomes '0' (ASCII 30H).
                  If AH is 1, it becomes '1' (ASCII 31H).
                  ; This is necessary because we want to display
                  these bits as characters.
    MOV BYTE PTR[SI], AH
              ; Moves the ASCII binary digit (from AH)
              to the memory location pointed to by SI (within STR1).
              ; These digits are stored in **reverse order**
              (Least Significant Bit (LSB) first) because remainders
              are generated from LSB to MSB.
    MOV AH, 00 ; Clears the AH register by setting it to 0.
              ; This is crucial before the next DIV operation.
              The DIV instruction uses the AH register as the upper 8 bits
              of the dividend (number being divided) for 16-bit division.
              ; Clearing it ensures the next division operates correctly
              on the AL value.
    INC SI     ; Increments the SI register by 1, making it point
              to the next available position in STR1 for the next binary digit.
    INC BH     ; Increments the BH register (binary digit counter) by 1.
    CMP AL, 00 ; Compares the quotient (in AL) with 0.
    JNE L1     ; If the quotient is not 0, it means there are still
              values to divide.
              Jumps back to the L1 loop to continue the division process.
              ; The loop terminates when the quotient becomes 0,
              indicating all binary bits have been extracted.
```

The **L1** loop implements the core decimal-to-binary conversion. It repeatedly uses the **DIV BL** instruction to divide the current number in **AX** by 2. The remainder is converted

to ASCII and stored in **STR1**. A crucial aspect here is that the **quotient of this division is stored in the 8-bit AL register**. This inherent **8-bit limit of AL (maximum value 255)** means that if the quotient exceeds 255, an overflow will occur, leading to incorrect results. This is why the program effectively handles decimal inputs only up to 511 (since $511/2=255$ with a remainder). **BH** tracks the total number of bits, and the loop continues as long as **AL** (the quotient) is not zero.

2.3.9. Reverse STR1 to STR2

Since the binary digits are stored in **STR1** in reverse order (LSB first), this stage is necessary to **reverse the string** and store it in **STR2** so that it can be displayed correctly (MSB first).

```

MOV CL,BH      ; Copies the total count of binary digits
                ; (from BH, e.g., 4 for 1010) into the CL register.
                ; This value will serve as the loop counter
                ; for the LOOP instruction later.

LEA SI,STR1     ; Resets SI to point to the beginning address of the STR1 buffer.
                ; This prepares SI to start reading from STR1 again,
                ; but this time in reverse.

LEA DI,STR2     ; Loads the effective address of the STR2 buffer
                ; (where the final, correctly ordered binary string will be stored)
                ; into the DI (Destination Index) register.
                ; DI will be used as a pointer to the current
                ; writing position within STR2.

MOV CH,00      ; Clears the CH register to 0.
                ; (CH and CL together form the 16-bit CX register).
                ; The LOOP instruction uses the full CX register,
                ; so even if only CL has a value, CH should be cleared.

ADD SI,CX       ; Adds the value of CX (the total number of binary digits) to SI.
                ; This moves SI from the beginning of STR1 to the memory location
                ; just after the last stored valid binary digit.
                ; For example, if STR1 was [0][1][0][1][ ]...,
                ; SI would point to the empty space after the last '1'.

DEC SI         ; Decrements SI by 1. This makes SI point directly to the
                ; last valid binary digit stored in STR1.
                ; This digit is actually the Most Significant Bit (MSB)
                ; of the binary number (e.g., the first '1' in '1010').

L2:            ; Marks the beginning of the loop for reversing the string
                ; from STR1 to STR2.

MOV AH,BYTE PTR[SI]
                ; Loads the character at the memory location pointed to by SI

```

```

; (which is reading STR1 from right-to-left) into the AH register.
; This effectively reads the binary bits starting from the
; MSB (the "first" bit of the actual binary number).
MOV BYTE PTR[DI],AH
; Moves the character from AH to the memory location
; pointed to by DI (which is writing to STR2 from left-to-right).
DEC SI
; Decrements SI by 1. This moves SI backward in STR1,
; effectively reading the next bit towards the LSB.
INC DI
; Increments DI by 1. This moves DI forward in STR2,
; preparing for the next bit to be written.
LOOP L2
; Decrements the CX register by 1. If CX is not zero,
; jumps back to L2.
; This loop continues for CL (total digit count) iterations,
; ensuring all bits are copied and reversed.

```

The **L2** loop is crucial for presenting the binary number in the correct order. Since **L1** stores bits in reverse, **L2** addresses this by reading **STR1** from the last written bit (which is the MSB) backward towards the first written bit (LSB), and simultaneously writing these bits into **STR2** from left to right. **SI** moves backward through **STR1**, and **DI** moves forward through **STR2**. The **LOOP** instruction ensures this reversal process occurs for every extracted bit.

2.3.10. Display Binary Result

This combined stage is responsible for presenting the final binary conversion result to the user in a clear and user-friendly manner, including a newline, a descriptive message, and the actual binary string.

```

DIS LINE
; Calls the DIS macro to display the newline characters
; (Carriage Return and Line Feed) defined in the LINE variable.
; This creates a visual separation between previous output/input
; prompts and the upcoming binary result, improving readability.
DIS MSG2
; Calls the DIS macro to display the "BINARY NUMBER IS : " message
; defined in MSG2.
; This message acts as a label, clearly indicating to the user
; what the following number represents.
DIS STR2
; Calls the DIS macro to display the final,
; correctly ordered binary string stored in STR2.
; This string is the binary representation of the decimal number
; entered by the user and is the main output of the program.

```

This integrated **result display stage** completes the user experience. **DIS LINE** ensures clean formatting by starting the output on a new line. **DIS MSG2** provides a clear label

for the output. Finally, **DIS STR2** displays the actual binary string, which has been accurately converted and correctly ordered in the **STR2** buffer, fulfilling the primary purpose of the program.

2.3.11. Program End

After all operations are complete, this stage ensures that the program terminates gracefully and returns control to the operating system.

```
MOV AH,4CH    ; Loads AH register with 4CH, which is the DOS service number
               ; for program termination.
               ; This is a standard request to the operating system
               ; to end the current program.
INT 21H       ; Calls DOS Interrupt 21H. This executes the 4CH service,
               ; terminating the program's execution and returning control
               ; to the operating system (e.g., DOSBox).
CODE ENDS     ; Marks the end of the code segment.
END INPUT     ; Informs the assembler that the 'INPUT' label is the
               ; starting point (entry point) of the program's execution.
```

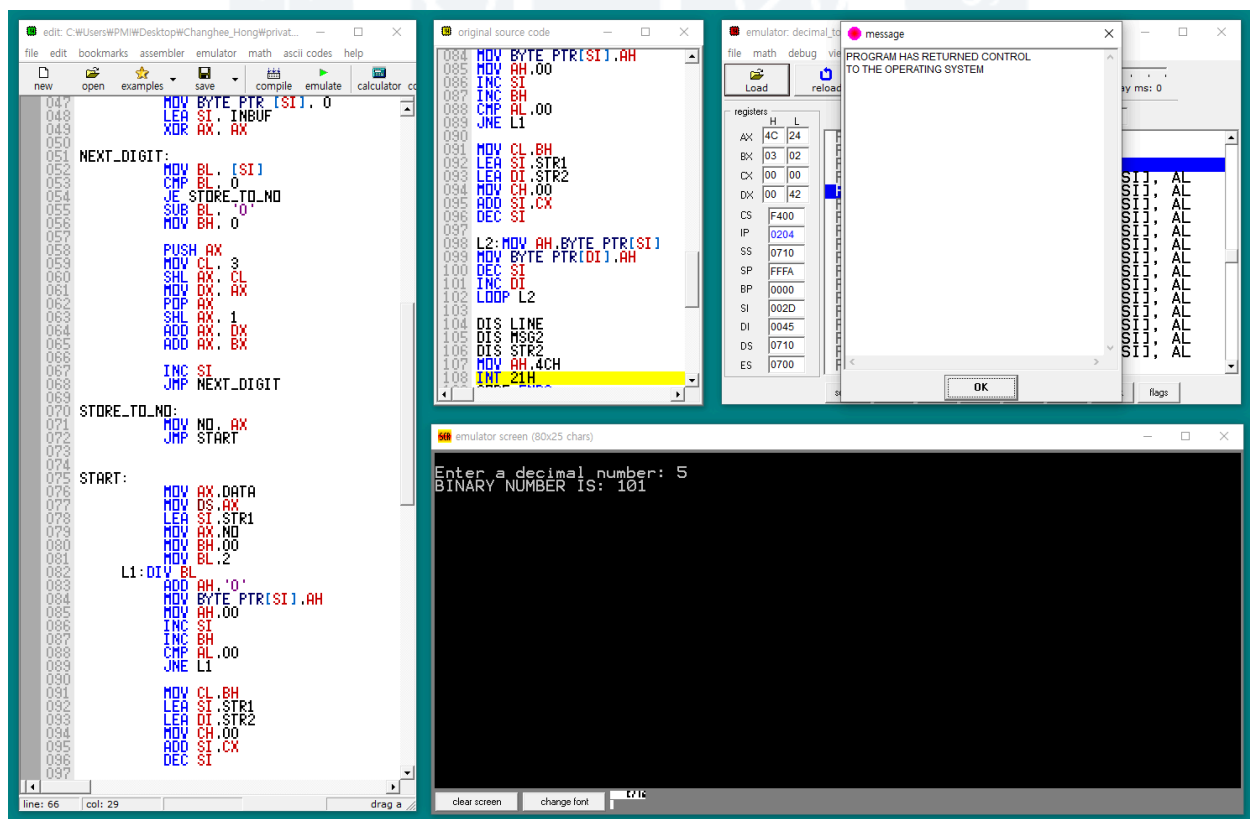
The **MOV AH, 4CH** and **INT 21H** instructions are the standard way to terminate a program correctly in a DOS environment. This ensures that the program releases any resources it used and returns control to the operating system. **CODE ENDS** signifies the physical end of the code segment, and **END INPUT** is a crucial directive that tells the assembler where the program's execution should begin.

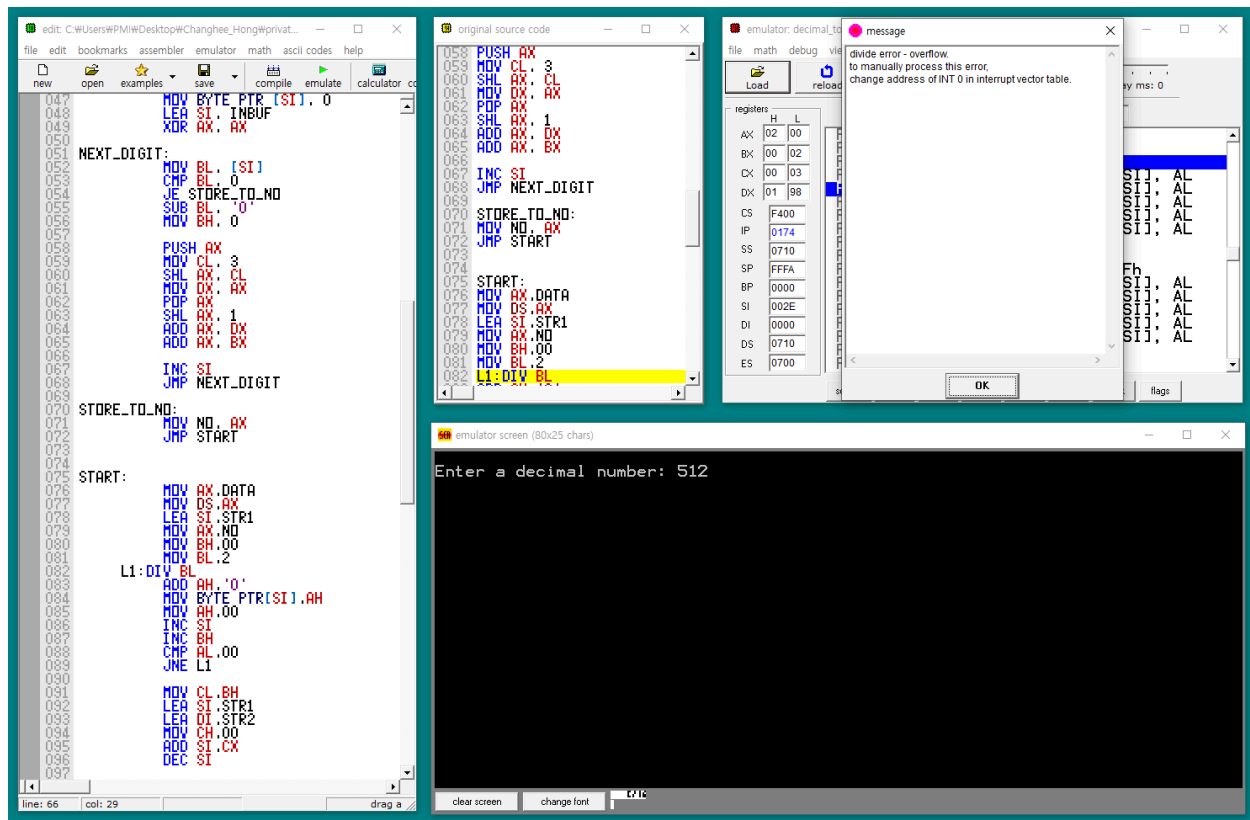
2.4. Project Results

The project successfully delivered a functional decimal-to-binary converter, demonstrating key assembly language concepts.

- **Correct Conversion within Limits:** The program accurately converts positive decimal numbers into their 2-byte binary representations. However, as a significant finding discussed in Section 4, the practical conversion limit for this implementation is **up to 511**. Inputs exceeding this value may lead to incorrect results due to underlying architectural constraints of the 16-bit 8086 processor's **DIV** instruction.
- **User Interaction:** The application provides clear prompts and handles user input gracefully. It includes basic validation to ensure only numeric characters are accepted, enhancing user experience.
- **Output Clarity:** The final output is well-formatted, clearly displaying the converted binary result to the user.

Figure 2.4.1: Screenshots of Program Execution





3. Tool and Software Used

The development and execution of this Decimal to Binary Converter were carried out using the following tools and software:

- EMU8086 Integrated Development Environment (IDE) and Emulator:**
 EMU8086 served as the primary platform for this project. It offered a comprehensive environment for writing, assembling, and debugging the 8086 Assembly Language code. Its integrated assembler translated the source code into executable machine instructions, while the powerful emulator accurately simulated the behavior of the 8086 microprocessor. Crucially, EMU8086's visual debugging features, such as register viewing, memory inspection, and step-by-step execution, were indispensable for understanding the low-level program flow, identifying logical errors, and fine-tuning the code. This environment allowed for a direct and intimate interaction with the CPU's operations, which was central to the learning experience.

4. Meaningful Benefits and Findings

This project provided invaluable insights into low-level programming and the direct interaction with CPU architecture. Through the implementation of a Decimal to Binary Converter in Assembly Language, a profound understanding of fundamental computer operations, including character-to-integer conversion, repetitive division, and string manipulation, was gained. The process highlighted the efficiency achievable through direct hardware control and optimized algorithms, such as the bit-shift-based multiplication by 10.

However, a particularly **meaningful finding** during the development and testing phase was the discovery of a specific **limitation inherent to the 16-bit architecture** (e.g., 8086/8088 CPU) emulated by EMU8086. It was observed that the program accurately converts decimal numbers only when the input value is **less than 512**.

This limitation stems from the **8-bit nature of the AL register**, which stores the quotient of the **DIV BL** (divide by 2) instruction. An 8-bit register can only represent values up to $2^8 - 1 = 255$. When a decimal number like 512 is divided by 2, the quotient becomes 256. This value exceeds **AL**'s capacity, leading to an **arithmetic overflow** and incorrect program behavior. For instance, $511/2$ results in a quotient of 255 (fitting **AL**), but $512/2$ yields 256, which **AL** cannot properly store.

This finding underscores the critical importance of understanding **register sizes and the precise mechanics of CPU instructions** in assembly programming. It serves as a powerful reminder that direct hardware manipulation requires meticulous attention to data representation limits. Overcoming such limitations in future projects would necessitate transitioning to a 32-bit (e.g., using **EAX**, **EBX**) or 64-bit environment, or implementing more complex **multi-precision arithmetic** techniques in a 16-bit setting. This experience deepened my appreciation for the complexities managed by higher-level programming languages and compilers.